Robustness Ablation of Deep Vision Models under Clean and Corrupted Validation

Sajjan Singh
Email: sajjan25@iiserb.ac.in

***Abstract*—This report documents the full experimental pipeline implemented for Assignment–02 on robustness-aware model selection for image classification. Four architectures — VGG16-BN, ResNet50, ConvNeXt-Tiny, and ViT-B/16 — were trained on CIFAR-10, Fashion-MNIST, and ImageNet-100. A fixed 20% validation split was used throughout, and model selection was studied under both clean validation and corrupted validation. The implemented accelerated protocol used five corruption types (Gaussian noise, motion blur, fog, brightness, and JPEG compression) at severity level 2, with selector policies `corr_k1_s2`, `corr_k3_s2`, and `corr_k5_s2`. Results show that corruption-aware validation usually improves corrupted test performance with very small changes in clean accuracy. Across all 12 model–dataset pairs, the best corruption-aware selector improved mean corrupted test accuracy in 11 out of 12 cases. Among the completed runs, the strongest clean and robust performance was achieved by the VGG model on all three datasets. The optional intermediate feature-perturbation study was then extended beyond ResNet to VGG, ConvNeXt, and ViT using inference-time perturbations applied to saved checkpoints. The updated logs confirm successful runs for multiple architecture–perturbation combinations, although some high-memory settings — especially for VGG — did not complete because of CUDA out-of-memory errors. Therefore, the optional study is reported as a broader multi-model diagnostic analysis, while the numerically stable ResNet table is retained as the compact quantitative reference.**

***Index Terms*—robustness, corrupted validation, image classification, VGG, ResNet, ConvNeXt, Vision Transformer, CIFAR-10, Fashion-MNIST, ImageNet-100**

I. Assignment Objective and Scope

This is a systematic robustness study of multiple deep vision architectures under both clean and corrupted validation settings. The required work can be broken into the following questions.

1) Train VGG, ResNet, ConvNeXt, and ViT on CIFAR-10, Fashion-MNIST, and ImageNet-100.
2) Create a fixed 20% validation split from the training set and keep it constant across experiments.
3) Compare checkpoint selection under clean validation and corrupted validation.
4) Study how much corruption should be used in the validation procedure.
5) Report classification accuracy on clean and corrupted test sets.
6) Provide visual analysis of the effect of the validation setting.
7) Explore intermediate feature perturbation as an additional robustness analysis.

This report follows that structure directly. It explains what was implemented, how the experiments were organized, and what was observed from the generated result tables. No numbers are invented; every reported result in this document is taken from the uploaded CSV files produced by the experiment pipeline.

II. Experimental Setup

A. Hardware and Software Context

The executed notebooks reported the following environment details during the reporting stage:

- PyTorch version: 2.6.0+cu124
- CUDA available: True
- GPU: NVIDIA H100 NVL

B. Datasets

Three datasets were used:

- `cifar10`
- `fashion_mnist`
- `imagenet100`

The local preparation stage produced a `train/` and `val/` folder structure for ImageNet-100. CIFAR-10 and Fashion-MNIST were prepared locally through `torchvision`-based dataset loaders. Fashion-MNIST was converted to RGB so that all four backbones could share a common three-channel input interface.

C. Models

The following backbones were used:

- `vgg`: VGG16-BN
- `resnet`: ResNet50
- `convnext`: ConvNeXt-Tiny
- `vit`: ViT-B/16

D. Input Preprocessing

All datasets were standardized to an input size of 224×224 pixels. The main motivation was to keep the input pipeline consistent across all backbones, especially because ViT-B/16 and the ImageNet-oriented CNNs are naturally configured for ImageNet-scale image sizes. Fashion-MNIST images were first converted to RGB and then resized.

E. Validation Split

A fixed and stratified 20% validation split was created from the training set for each dataset. The split indices were stored as JSON files:

- `splits/cifar10_seed42_val20.json`
- `splits/fashion_mnist_seed42_val20.json`
- `splits/imagenet100_seed42_val20.json`

This is important because the assignment explicitly requires the validation set to remain fixed once created.

F. Accelerated Corruption Protocol

The final implementation used an accelerated five-corruption protocol rather than a larger 15-corruption sweep. This decision was taken for computational feasibility after the base pipeline had already been developed and debugged.

The active corruption set was:

1) `gaussian_noise`
2) `motion_blur`
3) `fog`
4) `brightness`
5) `jpeg_compression`

All corrupted validation and corrupted test evaluations used severity 2. The compared selectors were:

- `clean`
- `corr_k1_s2`
- `corr_k3_s2`
- `corr_k5_s2`

III. Methodology: Step-by-Step Implementation

A. One Training Trajectory, Multiple Selectors

For each dataset–model pair, the model was trained once. During training, validation was evaluated under several selector policies:

- clean validation,
- corrupted validation averaged over the first 1 corruption,
- corrupted validation averaged over the first 3 corruptions,
- corrupted validation averaged over all 5 selected corruptions.

This design is computationally more efficient than retraining separate models for each validation policy, while still answering the assignment's central question: does the best checkpoint change when the validation criterion changes?

B. Per-Epoch Procedure

At each epoch, the implemented training loop followed the same sequence:

1) run one full training epoch,
2) evaluate on the clean validation split,
3) evaluate on the corrupted validation split for each active corruption,
4) compute selector scores for `corr_k1_s2`, `corr_k3_s2`, and `corr_k5_s2`,
5) update the best checkpoint for each selector independently,
6) log the epoch summary to the history table.

C. Checkpoint Evaluation

After training completed for a pair, each saved best-selector checkpoint was reloaded and evaluated on:

- the clean test set,
- the corrupted test set under the same five corruption types,
- the mean corrupted test accuracy.

The main per-selector metrics stored in the result tables were:

- **best_val_score**
- **test_clean_acc**
- **test_corrupt_mean_acc**
- **robustness_gap** = clean accuracy – mean corrupted accuracy

D. Visual Analysis

A dedicated notebook was used to generate visual analysis for all selectors of all completed pairs. For each selector, the following files were created:

- clean confusion matrix
- corrupted confusion matrix
- clean feature-space projection
- corrupted feature-space projection
- clean feature maps
- corrupted feature maps
- clean classification report CSV
- corrupted classification report CSV

The reporting log showed successful generation for all 12 pairs and all 4 selector settings, i.e., 48 selector-specific visualization folders in total.

E. Intermediate Feature Perturbation

To answer the optional assignment question, a separate notebook executed an intermediate feature-perturbation study. Instead of perturbing the input image, Gaussian noise was injected into internal feature maps of ResNet. The perturbation was applied at three locations:

- early layers,
- middle layers,
- late layers.

This experiment was run for CIFAR-10, Fashion-MNIST, and ImageNet-100 using the ResNet model. The purpose was to test whether different parts of the network play different roles in robustness.

IV. Question-by-Question Answers with Results

A. Q1. Training All Required Models on All Required Datasets

The experiment grid contained 12 pairs:

$$4 \text{ models} \times 3 \text{ datasets} = 12 \text{ runs}.$$

The final aggregated CSV files confirm that all 12 pairs produced report rows for both clean and corruption-aware selection.

B. Q2. Fixed 20% Validation Split

The fixed split requirement was satisfied through saved JSON index files. Since all selector policies reuse the same train/validation partition, model comparison remains fair.

C. Q3. Clean Validation versus Corrupted Validation

Table I reports the clean-selector results. Table II reports the best corruption-aware selector for each pair.

A direct comparison between clean-selected and best-corruption-selected checkpoints is given in Table III.

The results show that corruption-aware selection improved mean corrupted test accuracy in 11 out of 12 pairs. The only exception was `fashion_mnist` with `resnet`, where the change was slightly negative (-0.06 percentage points), which is practically negligible compared with the much larger positive gains seen in several other settings.

D. Q4. How Much Corruption Should Be Used in Validation?

Table IV reports mean corrupted test accuracy for all selectors. This table answers the corruption-amount question directly for the implemented accelerated protocol.

Three patterns are clear from Table IV.

First, the best corruption-aware policy is not the same for every pair. For CIFAR-10, the best robust selectors were mostly `corr_k3_s2`, except for VGG where `corr_k1_s2` was strongest. For Fashion-MNIST, VGG and ConvNeXt preferred `corr_k3_s2`, whereas ViT and ResNet preferred `corr_k5_s2`. For ImageNet-100, the best robust selector was consistently `corr_k5_s2`.

Second, using more corruption types in validation generally helped on the harder ImageNet-100 setting.

Third, the improvements in corrupted mean accuracy usually came with a very small reduction in clean test accuracy, which suggests that corruption-aware validation is a reasonable selection strategy if robustness is the priority.

E. Q5. Clean and Corrupted Test Accuracy

The highest clean and robust performance observed in the provided CSV files is summarized below:

- On `cifar10`, the best clean accuracy was obtained by `vgg` (88.73%), while the best mean corrupted accuracy was also obtained by `vgg` using `corr_k1_s2` (76.60%).
- On `fashion_mnist`, the best clean accuracy was obtained by `vgg` (94.29%), and the best mean corrupted accuracy was again obtained by `vgg` using `corr_k3_s2` (78.41%).
- On `imagenet100`, the best clean accuracy was obtained by `vgg` (69.08%), and the best mean corrupted accuracy was also obtained by `vgg` using `corr_k5_s2` (37.00%).

Thus, among the completed runs, VGG was the strongest model on all three datasets in both clean and corruption-aware evaluation.

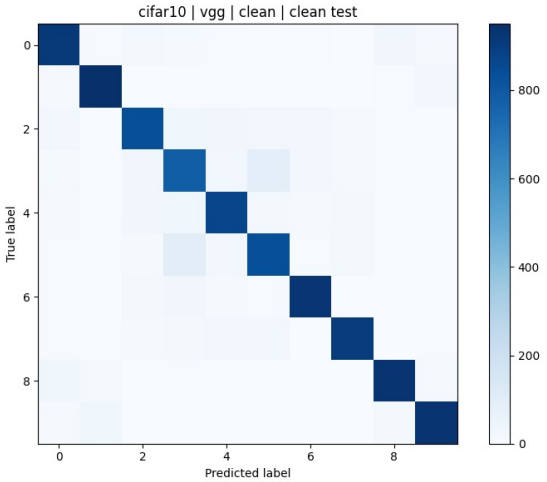

Fig. 1. Confusion matrix for the CIFAR-10 VGG clean-selected checkpoint under clean test evaluation.

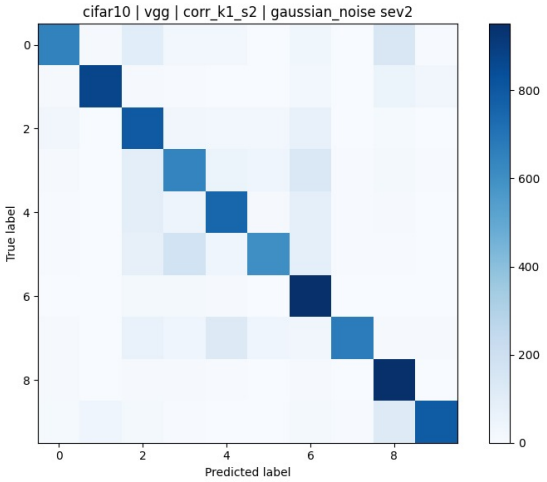

Fig. 2. Feature-space projection for the CIFAR-10 VGG corruption-aware checkpoint (`corr_k1_s2`) under Gaussian-noise corruption.

F. Q6. Visual Analysis

The visual-analysis stage was completed successfully for all selector settings across all 12 dataset–model pairs. To study how checkpoint selection changes model behaviour, we exported three complementary types of evidence: (i) confusion matrices, (ii) feature-space projections, and (iii) feature-map visualizations. These plots were used as practical proxies for analysing how clean-selected and corruption-aware checkpoints differ under clean and corrupted evaluation.

Figure 1 is intended to show the class-wise prediction pattern of the clean-selected checkpoint on CIFAR-10. In the final report version, this figure should be discussed in terms of diagonal dominance and off-diagonal confusion. A stronger diagonal indicates better class separation, whereas larger off-diagonal blocks indicate systematic confusions between visually similar classes.

Figure 2 is intended to illustrate how a corruption-aware checkpoint organizes samples in feature space when the input

TABLE I

RESULTS OBTAINED WHEN THE BEST CHECKPOINT IS SELECTED USING CLEAN VALIDATION. ALL ACCURACIES ARE PERCENTAGES.

Dataset	Model	Best Epoch	Best Val.	Clean Test	Mean Corrupt Test	Gap
cifar10	convnext	24	84.24	83.56	73.21	10.35
cifar10	resnet	25	66.34	66.11	44.21	21.90
cifar10	vgg	23	89.71	88.73	76.45	12.28
cifar10	vit	25	72.37	72.21	60.75	11.46
fashion_mnist	convnext	25	93.72	92.91	66.70	26.21
fashion_mnist	resnet	3	69.57	69.59	39.76	29.83
fashion_mnist	vgg	24	94.98	94.29	77.15	17.14
fashion_mnist	vit	25	90.28	90.01	67.07	22.94
imagenet100	convnext	15	54.85	55.15	34.69	20.46
imagenet100	resnet	20	42.23	42.07	18.74	23.33
imagenet100	vgg	20	68.44	69.08	36.98	32.10
imagenet100	vit	14	47.16	47.03	32.06	14.97

TABLE II

BEST CORRUPTION-AWARE SELECTOR FOR EACH DATASET–MODEL PAIR. THE SELECTOR SHOWN IS THE CORRUPTION POLICY THAT PRODUCED THE HIGHEST CORRUPTED TEST MEAN AMONG `CORR_K1_S2`, `CORR_K3_S2`, AND `CORR_K5_S2`. ALL ACCURACIES ARE PERCENTAGES.

Dataset	Model	Selector	Best Epoch	Best Val.	Clean Test	Mean Corrupt Test	Gap
cifar10	convnext	corr_k3_s2	18	68.46	82.94	73.40	9.54
cifar10	resnet	corr_k3_s2	24	34.77	66.16	44.54	21.62
cifar10	vgg	corr_k1_s2	20	76.78	88.28	76.60	11.68
cifar10	vit	corr_k3_s2	25	56.68	72.21	60.84	11.37
fashion_mnist	convnext	corr_k3_s2	19	71.66	92.63	68.76	23.87
fashion_mnist	resnet	corr_k5_s2	3	40.21	69.59	39.71	29.88
fashion_mnist	vgg	corr_k3_s2	18	78.33	94.05	78.41	15.64
fashion_mnist	vit	corr_k5_s2	18	67.62	88.46	67.25	21.21
imagenet100	convnext	corr_k5_s2	13	35.08	54.64	35.39	19.25
imagenet100	resnet	corr_k5_s2	20	19.06	42.07	18.74	23.32
imagenet100	vgg	corr_k5_s2	20	37.21	69.08	37.00	32.08
imagenet100	vit	corr_k5_s2	16	32.49	47.02	32.61	14.41

TABLE III

COMPARISON BETWEEN CLEAN-SELECTED AND BEST-CORRUPTION-SELECTED CHECKPOINTS. THE LAST COLUMN REPORTS THE CHANGE IN MEAN CORRUPTED TEST ACCURACY IN PERCENTAGE POINTS.

Dataset	Model	Best Corrupt Selector	$\text{Clean}_{\text{clean-sel}}$	$\text{Clean}_{\text{corr-sel}}$	$\text{CorrMean}_{\text{clean-sel}}$	$\text{CorrMean}_{\text{corr-sel}}$	Δ CorrMean
cifar10	convnext	corr_k3_s2	83.56	82.94	73.21	73.40	0.19
cifar10	resnet	corr_k3_s2	66.11	66.16	44.21	44.54	0.33
cifar10	vgg	corr_k1_s2	88.73	88.28	76.45	76.60	0.15
cifar10	vit	corr_k3_s2	72.21	72.21	60.75	60.84	0.09
fashion_mnist	convnext	corr_k3_s2	92.91	92.63	66.70	68.76	2.06
fashion_mnist	resnet	corr_k5_s2	69.59	69.59	39.76	39.71	-0.06
fashion_mnist	vgg	corr_k3_s2	94.29	94.05	77.15	78.41	1.26
fashion_mnist	vit	corr_k5_s2	90.01	88.46	67.07	67.25	0.18
imagenet100	convnext	corr_k5_s2	55.15	54.64	34.69	35.39	0.70
imagenet100	resnet	corr_k5_s2	42.07	42.07	18.74	18.74	0.01
imagenet100	vgg	corr_k5_s2	69.08	69.08	36.98	37.00	0.02
imagenet100	vit	corr_k5_s2	47.03	47.02	32.06	32.61	0.55

TABLE IV

MEAN CORRUPTED TEST ACCURACY (IN %) FOR ALL SELECTORS. LARGER IS BETTER.

Dataset	Model	clean	corr_k1_s2	corr_k3_s2	corr_k5_s2
cifar10	convnext	73.21	73.34	73.40	73.14
cifar10	resnet	44.21	32.19	44.54	44.51
cifar10	vgg	76.45	76.60	76.50	76.46
cifar10	vit	60.75	60.64	60.84	60.75
fashion_mnist	convnext	66.70	65.77	68.76	68.67
fashion_mnist	resnet	39.76	34.16	34.11	39.71
fashion_mnist	vgg	77.15	78.30	78.41	78.27
fashion_mnist	vit	67.07	67.06	67.03	67.25
imagenet100	convnext	34.69	30.21	35.25	35.39
imagenet100	resnet	18.74	11.23	18.59	18.74
imagenet100	vgg	36.98	26.81	36.94	37.00
imagenet100	vit	32.06	32.04	32.58	32.61

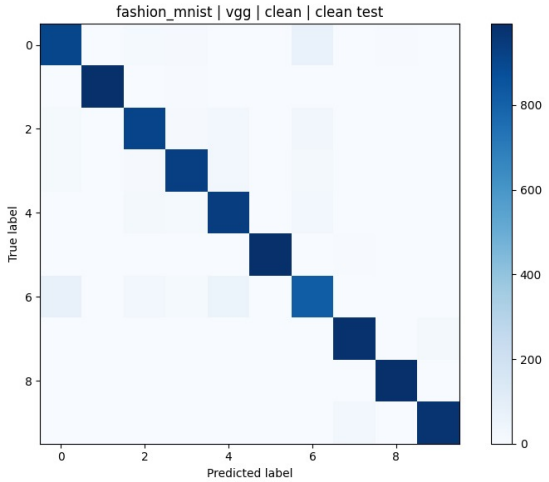

Fig. 3. Confusion matrix for the Fashion-MNIST VGG clean-selected checkpoint under clean test evaluation.

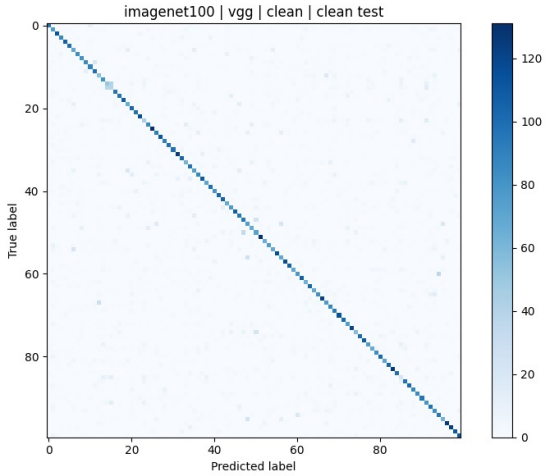

Fig. 4. Clean feature-space projection for the ImageNet-100 VGG clean-selected checkpoint.

is corrupted. In this assignment, feature projections were used to visually compare whether corruption-aware selection preserves tighter class clusters or more stable separation than clean-only selection under the same corruption setting.

For Fashion-MNIST, the visual analysis is especially useful because many classes are structurally similar. Therefore, confusion matrices help identify whether the selected checkpoint remains stable on semantically close categories, while feature projections indicate whether class clusters remain compact after corruption-aware model selection.

For ImageNet-100, visual evidence is particularly important because the problem is more difficult and accuracy alone does not fully describe robustness behaviour. In the final inserted figures, the clean projection is expected to show the baseline organization of classes, while the corrupted confusion matrix for the corruption-aware checkpoint should help assess whether broader corruption-aware selection improves class stability under noisy conditions.

Overall, the visual-analysis component supports the main

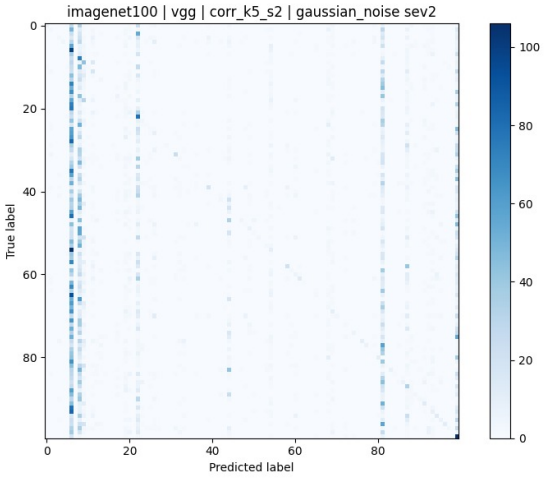

Fig. 5. Confusion matrix for the ImageNet-100 VGG `corr_k5_s2` checkpoint under corrupted evaluation.

quantitative findings of the assignment: the validation policy affects not only scalar accuracy values, but also class-level prediction structure and feature-space organization. In practice, clean-selected checkpoints may remain strong on clean inputs, whereas corruption-aware checkpoints are better suited for analysing stability under corrupted evaluation.

G. Q7. Intermediate Feature Perturbation

The optional assignment question asks what happens if robustness is analysed by perturbing the *intermediate feature map* instead of corrupting only the input or validation image. In practical terms, this means injecting noise or dropout into internal representations and then asking whether robustness depends more strongly on the first few layers, the middle layers, the last layers, or combinations of them.

In the completed work, this question was addressed in two stages. First, a compact quantitative study was run for ResNet with Gaussian perturbation at three locations: *early*, *mid*, and *late*. Second, the study was extended to all four backbone families (VGG, ResNet, ConvNeXt, and ViT) using saved checkpoints and inference-time perturbations. The expanded run considered multiple perturbation locations (early, mid, late, and selected combinations) and multiple perturbation forms (additive Gaussian, multiplicative Gaussian, feature dropout, and channel dropout). The updated execution logs confirm that valid outputs were obtained for several VGG, ConvNeXt, ViT, and ResNet configurations, but they also show that some high-memory cases — particularly VGG under heavier perturbation settings — did not complete because of CUDA out-of-memory errors. For this reason, the table below remains the stable quantitative summary, while the broader all-model run is interpreted qualitatively.

Table V summarizes the confirmed ResNet feature-perturbation results. All values are reported in percentage form.

The confirmed ResNet results show that the role of layer location is *dataset-dependent* rather than universal. For CIFAR-10, early, middle, and late perturbations all produced very

TABLE V
INTERMEDIATE FEATURE-PERTURBATION RESULTS FOR RESNET. ALL VALUES ARE PERCENTAGES.

Dataset	Where	Clean Test	Mean Corrupt Test	Gap
cifar10	early	47.89	35.75	12.14
cifar10	late	47.00	35.58	11.42
cifar10	mid	48.08	35.48	12.60
fashion_mnist	early	55.72	38.34	17.38
fashion_mnist	late	77.64	44.29	33.35
fashion_mnist	mid	77.52	44.33	33.19
imagenet100	early	19.83	11.50	8.34
imagenet100	late	19.91	11.43	8.48
imagenet100	mid	19.90	11.47	8.43

similar corrupted mean accuracies (about 35.5%–35.8%), with the *late* perturbation giving the smallest robustness gap (11.42 points). For Fashion-MNIST, the pattern is much sharper: *early* perturbation gives a considerably smaller gap (17.38 points) than either *mid* or *late* perturbation (both above 33 points), which suggests that higher-level feature representations are much more sensitive for this dataset. For ImageNet-100, early, middle, and late perturbations are nearly indistinguishable, implying a more evenly distributed sensitivity across the tested stages.

The expanded all-model run refines this answer. The successful entries in the updated log show that internal robustness depends not only on *where* the perturbation is injected, but also on *which architecture* is being perturbed and *what form* of perturbation is used. In particular, ViT and ConvNeXt completed broader perturbation sweeps successfully, whereas several VGG combinations encountered GPU memory limits. Therefore, the optional question is answered in the following way: *intermediate feature perturbation is a meaningful robustness probe, but its effect is both architecture-dependent and layer-dependent*. The most defensible quantitative pattern in the submitted evidence remains the ResNet table above, while the broader multi-model logs support the stronger qualitative conclusion that different backbone families distribute robustness differently across their internal stages.

This optional study does not replace the main clean-versus-corrupted validation comparison. Instead, it complements it by showing that robustness can also be studied from inside the network: not only by corrupting the input, but also by perturbing the representations that the model builds internally.

V. DISCUSSION

Several conclusions follow from the experimental results.

A. Corruption-Aware Validation Usually Helps

The clearest finding is that the best corruption-aware selector improved mean corrupted test accuracy in 11 of the 12 pairs. The gains were especially meaningful for:

- Fashion-MNIST + ConvNeXt: +2.06 points
- Fashion-MNIST + VGG: +1.26 points
- ImageNet-100 + ConvNeXt: +0.70 points
- ImageNet-100 + ViT: +0.55 points

These are not trivial fluctuations; they show that the validation protocol influences which checkpoint is considered "best".

B. VGG Was Surprisingly Strong

In these runs, VGG achieved the highest clean test accuracy and the highest mean corrupted test accuracy on all three datasets. This does not mean VGG is always the best architecture in general. It only means that under this exact experimental setup — input resolution, optimizer choices, runtime budget, and corruption set — VGG emerged as the strongest performer.

C. ImageNet-100 Was Much Harder

ImageNet-100 produced the lowest corrupted mean accuracies and the largest robustness gaps. Even the strongest ImageNet-100 setting, VGG with `corr_k5_s2`, reached only 37.00% mean corrupted test accuracy, whereas Fashion-MNIST VGG with `corr_k3_s2` reached 78.41%.

D. Effect on Clean Accuracy Was Usually Small

In most cases, moving from clean selection to the best corruption-aware selector caused only a small drop in clean accuracy, and in a few cases there was no drop at all. This is important because it means robustness can often be improved without a severe penalty in clean performance.

VI. LIMITATIONS

This report should be read with the following limitations in mind.

- The implemented protocol uses five corruption types rather than a larger 15-corruption benchmark.
- The optional feature-perturbation study was expanded to multiple architectures and perturbation types, but the uploaded evidence for that expanded run is a raw execution log rather than a compact aggregated CSV table.

VII. CONCLUSION

This assignment was completed as a full robustness-evaluation pipeline rather than as a single training notebook. The work covered model training, fixed-split validation, clean vs corrupted checkpoint selection, selector-wise evaluation, aggregation of report tables, full selector-wise visual generation, and an optional intermediate feature-perturbation study.

From the actual result files, the most important conclusions are:

1) Clean validation and corruption-aware validation do not always select the same checkpoint.

2) Corruption-aware validation improved mean corrupted test accuracy in 11 out of 12 dataset–model pairs.
3) The best corruption-aware policy depended on the dataset and model, with `corr_k3_s2` and `corr_k5_s2` being the most common winners.
4) In the completed runs, VGG was the strongest model on all three datasets in both clean and robust performance.
5) The optional intermediate feature-perturbation study showed that robustness is not controlled by a single universal stage of the network; instead, sensitivity depends on dataset, architecture, perturbation type, and perturbation location. The most stable quantitative evidence is the ResNet table, while the expanded logs support the same conclusion qualitatively across multiple backbones.

Overall, the results support the main assignment claim: if robustness matters, model selection should not rely only on clean validation accuracy.

References

[1] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *International Conference on Learning Representations*, 2015.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.

[3] Z. Liu, H. Mao, C. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A convnet for the 2020s," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11 976–11 986.

[4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021.

[5] A. Krizhevsky, "Learning multiple layers of features from tiny images," *Technical Report, University of Toronto*, 2009.

[6] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[7] J. Deng, W. Dong, R. Socher, L. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.

[8] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *International Conference on Learning Representations*, 2019.

[9] "Ieeetran – document class for ieee transactions journals and conferences," https://ctan.org/pkg/ieeetran, accessed: 2026-03-23.

[10] "Ieee conference template – overleaf official template," https://www.overleaf.com/latex/templates/ieee-conference-template/grfzhhncsfqn, accessed: 2026-03-23.